CSC D18 – Assignment 2, Part 1: Projection Mapping

Question 1

Setup. The observer camera is at the world origin, $\mathbf{e} = (0, 0, 0)$. Let $\mathbf{g}$ be the camera's gaze direction, $\mathbf{t}$ its up vector, and $f > 0$ its focal length (these are not specified in the question, so we keep them general). The sphere has center $\mathbf{c} = (c_x, c_y, c_z)$ and radius r.

Camera coordinate frame.

$$\mathbf{w} = -\frac{\mathbf{g}}{\|\mathbf{g}\|}, \qquad \mathbf{u} = \frac{\mathbf{t} \times \mathbf{w}}{\|\mathbf{t} \times \mathbf{w}\|}, \qquad \mathbf{v} = \mathbf{w} \times \mathbf{u}.$$

Pixel location in camera coordinates. The image plane window has size 2 in both x and y, and the image is 500×500 pixels, so each pixel has width and height $\frac{2}{500} = \frac{1}{250}$. The left edge of the window is at $x = -1$ and the top edge is at $y = 1$. Following the convention in the lecture notes, $i, j \in \{0, 1, \ldots, 499\}$, and we take the location of pixel (i, j) to be its top-left corner:

$$x = -1 + \frac{i}{250}, \qquad y = 1 - \frac{j}{250},$$

The image plane lies a distance f from the camera along the gaze direction. Since $\mathbf{w}$ points opposite to $\mathbf{g}$, the image plane is at $w = -f$ in camera coordinates. In homogeneous camera coordinates, the pixel is

$$\mathbf{p}_c = \begin{bmatrix} -1 + \frac{i}{250} \\ 1 - \frac{j}{250} \\ -f \\ 1 \end{bmatrix}.$$

Converting to world coordinates. The camera-to-world transformation matrix is

$$M_{cw} = \begin{bmatrix} u_x & v_x & w_x & e_x \\ u_y & v_y & w_y & e_y \\ u_z & v_z & w_z & e_z \\ 0 & 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} u_x & v_x & w_x & 0 \\ u_y & v_y & w_y & 0 \\ u_z & v_z & w_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix},$$

since $\mathbf{e} = \mathbf{0}$. The pixel in world coordinates is then

$$\mathbf{p}_w = M_{cw}\, \mathbf{p}_c.$$

Ray through the pixel. The ray starts at $\mathbf{e}$ with direction $\mathbf{d} = \mathbf{p}_w - \mathbf{e} = \mathbf{p}_w$. Since $\mathbf{e} = \mathbf{0}$, the ray is simply

$$\mathbf{r}(\lambda) = \lambda \mathbf{d}.$$

Ray–sphere intersection. The ray hits the sphere when

$$\|\lambda\mathbf{d} - \mathbf{c}\| = r.$$

From the lecture notes, the solutions are

$$\lambda = -\frac{B}{A} \pm \frac{\sqrt{D}}{A},$$

where

$$A = \mathbf{d} \cdot \mathbf{d}, \qquad B = -\mathbf{c} \cdot \mathbf{d}, \qquad C = \mathbf{c} \cdot \mathbf{c} - r^2, \qquad D = B^2 - AC.$$

If $D < 0$ there is no intersection. Otherwise, take λ^* to be the smallest positive solution, since only points in front of the camera can be seen. Plugging it back into the ray equation gives

$$\mathbf{i} = \lambda^*\mathbf{d},$$

the first intersection point between the ray and the sphere. This is the location on the projection surface that produces the colour seen by the camera at pixel $(i, j)_{\text{cam}}$.

Question 2

In Question 1 we converted from camera coordinates to world coordinates. Here we go the other way, from world coordinates to the coordinates of the projector, which acts as a camera. We assume the projector's image plane has the same window size (2×2) and resolution (500×500 pixels) as the observer camera's, since the question does not specify them.

Projector coordinate frame. The projector is gazing at the center of the sphere, so its gaze direction is

$$\mathbf{g}_p = \mathbf{c} - \mathbf{e}_p.$$

The projector's frame $\mathbf{u}_p, \mathbf{v}_p, \mathbf{w}_p$ is computed from $\mathbf{g}_p$ and the up vector $\mathbf{t} = (0, 1, 0)$ exactly as in Question 1.

World-to-projector transformation. Let $M_{pw} = \begin{bmatrix}\mathbf{u}_p & \mathbf{v}_p & \mathbf{w}_p\end{bmatrix}$ be the 3×3 matrix whose columns are the projector's basis vectors. The full projector-to-world matrix has the same form as in Question 1, except that the center of projection is no longer at the origin:

$$\hat{M}_{pw} = \begin{bmatrix} M_{pw} & \mathbf{e}_p \\ \mathbf{0}^T & 1 \end{bmatrix} = \begin{bmatrix} u_{p,x} & v_{p,x} & w_{p,x} & e_x \\ u_{p,y} & v_{p,y} & w_{p,y} & e_y \\ u_{p,z} & v_{p,z} & w_{p,z} & e_z \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

Since $\mathbf{u}_p, \mathbf{v}_p, \mathbf{w}_p$ are orthonormal, M_{pw} is an orthonormal matrix, so its inverse is simply its transpose:

$$M_{wp} = M_{pw}^{-1} = M_{pw}^T.$$

The translation must also be expressed in projector coordinates, so it becomes $-M_{wp}\,\mathbf{e}_p$. The full world-to-projector matrix is therefore

$$\hat{M}_{wp} = \begin{bmatrix} M_{wp} & -M_{wp}\,\mathbf{e}_p \\ \mathbf{0}^T & 1 \end{bmatrix},$$

and the surface point in projector coordinates is

$$\mathbf{s} = \hat{M}_{wp}\,\mathbf{p}_s.$$

Ray from the surface point to the projector. In projector coordinates, the projector's center of projection is at the origin. The ray from $\mathbf{s}$ to the center of projection is therefore

$$\mathbf{r}_p(\mu) = \mathbf{s} + \mu\,\mathbf{d}_p, \qquad \mathbf{d}_p = (0,0,0) - \mathbf{s} = -\mathbf{s}.$$

Intersection with the projector's image plane. The image plane's normal is $\mathbf{w}_p$, which we also need in projector coordinates:

$$\mathbf{n}_p = M_{wp}\,\mathbf{w}_p.$$

The plane passes through $(0,0,-f)$, so its equation is

$$\big(\mathbf{x} - (0,0,-f)\big) \cdot \mathbf{n}_p = 0.$$

Substituting the ray, we want

$$\big(\mathbf{s} + \mu\,\mathbf{d}_p - (0,0,-f)\big) \cdot \mathbf{n}_p = 0,$$

which holds when

$$\mu^* = \frac{\big((0,0,-f) - \mathbf{s}\big) \cdot \mathbf{n}_p}{\mathbf{d}_p \cdot \mathbf{n}_p}.$$

The point on the projector's image plane is then $\mathbf{q} = \mathbf{r}_p(\mu^*)$.

Projection as a matrix. Equivalently, the projection onto the image plane can be written as a matrix acting on the homogeneous point $\mathbf{s} = (s_x, s_y, s_z, 1)$:

$$P = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & -\frac{1}{f} & 0 \end{bmatrix}, \qquad P\,\mathbf{s} = \begin{bmatrix} s_x \\ s_y \\ s_z \\ -\frac{s_z}{f} \end{bmatrix}.$$

The result is the same vector, except that the homogeneous coordinate is now $-\frac{s_z}{f}$. Dividing through by it (i.e. multiplying every component by $-\frac{f}{s_z}$) gives the projected point

$$\mathbf{q} = \left(-\frac{f\,s_x}{s_z},\ -\frac{f\,s_y}{s_z},\ -f \right).$$

Putting everything together, the full chain of matrices taking the surface point to the projector's image plane is

$$P\,\hat{M}_{wp}\,\mathbf{p}_s,$$

followed by the division by the homogeneous coordinate.

Pixel coordinates. To find the projector pixel (i_p, j_p) that $\mathbf{q}$ falls in, we only need q_x and q_y. Using the same top-left-corner convention as in Question 1, and rounding down so that any point inside a pixel maps to that pixel,

$$q_x = -1 + \frac{i_p}{250} \implies i_p = \big\lfloor (q_x + 1) \cdot 250 \big\rfloor,$$

$$q_y = 1 - \frac{j_p}{250} \implies j_p = \big\lfloor (1 - q_y) \cdot 250 \big\rfloor.$$

If i_p or j_p falls outside the valid range $\{0, 1, \ldots, 499\}$, then $\mathbf{q}$ is not in the projector's image, so there is nothing for the projector to project onto this surface point.

Question 3

The procedure below maps every pixel in the camera's image plane to the corresponding pixel in the projector's image plane. It works for any projection surface, because it uses each object's own intersection function from the ray tracer rather than a formula for a specific shape.

Here $\mathbf{e}$ is the camera's center of projection, f_c is the camera's focal length, $\hat{M}_{cw}$ is the camera-to-world matrix from Question 1, and $\mathbf{e}_p$, $\hat{M}_{wp}$ and P are the projector's center of projection, world-to-projector matrix and projection matrix from Question 2. MATVECMULT is the matrix–vector multiplication function from the starter code. FINDFIRSTHIT(*ray*) is the ray tracer's function that returns the λ and the point of the closest intersection between the ray and any object in the scene, with $\lambda = -1$ if the ray hits nothing. For the occlusion test we use every intersection between the ray and each object (e.g. both roots of the quadratic for a sphere), not only the closest one. $\varepsilon > 0$ is a small tolerance, and NONE indicates that no mapping can be constructed for the pixel.

Mapping a single pixel.

function CAMERATOPROJECTOR(i, j)
    // Convert the camera pixel to world coordinates (Question 1)
    $x \leftarrow -1 + \frac{i}{250}$
    $y \leftarrow 1 - \frac{j}{250}$
    $\mathbf{p}_{\text{world}} \leftarrow \text{MATVECMULT}\big(\hat{M}_{cw}, (x, y, -f_c, 1)\big)$

    // Find the surface point seen through this pixel
    $\mathbf{r}_c(\lambda) \leftarrow \mathbf{e} + \lambda\,(\mathbf{p}_{\text{world}} - \mathbf{e})$
    $(\lambda^*, \mathbf{x}^*) \leftarrow \text{FINDFIRSTHIT}(\mathbf{r}_c)$
    if $\lambda^* = -1$ **then**
        return NONE ▷ the ray hits nothing
    end if
    $\mathbf{p}_{\text{world}} \leftarrow (\mathbf{x}^*, 1)$

    // Check that the projector's light can reach the surface point
    $\mathbf{r}_s(\lambda) \leftarrow \mathbf{p}_{\text{world}} + \lambda\,(\mathbf{e}_p - \mathbf{p}_{\text{world}})$
    // A hit blocks the light only if it lies strictly between the surface point
    // and the projector, i.e. $\varepsilon < \lambda < 1$ (at $\lambda = 1$ we are at $\mathbf{e}_p$).
    // Hits with $\lambda \le \varepsilon$ are the ray re-hitting the surface it starts on.
    for all objects *obj* in the scene **do**
        for all intersections λ of *obj* with $\mathbf{r}_s$ **do**
            if $\varepsilon < \lambda < 1$ **then**
                return NONE ▷ something blocks the projector
            end if
        end for
    end for

    // Convert the world point to the projector's image plane (Question 2)
    $\mathbf{p}_{\text{proj}} \leftarrow \text{MATVECMULT}\big(P, \text{MATVECMULT}(\hat{M}_{wp}, \mathbf{p}_{\text{world}})\big)$
    $\mathbf{p}_{\text{proj}} \leftarrow \frac{1}{p_{\text{proj},h}}\,\mathbf{p}_{\text{proj}}$ ▷ $p_{\text{proj},h}$ is the homogeneous coordinate
    $i_p \leftarrow \big\lfloor (p_{\text{proj},x} + 1) \cdot 250 \big\rfloor$
    $j_p \leftarrow \big\lfloor (1 - p_{\text{proj},y}) \cdot 250 \big\rfloor$

```
    if i_p ∉ {0, ..., 499} or j_p ∉ {0, ..., 499} then
        return None                                   ▷ outside the projector's image
    end if
    return (i_p, j_p)
end function
```

Mapping every pixel.

```
function CameraToProjectorAll( )
    mapping ← a 500 × 500 matrix
    for i = 0 to 499 do                               ▷ width of the image plane in pixels
        for j = 0 to 499 do                           ▷ height of the image plane in pixels
            mapping[i][j] ← CameraToProjector(i, j)
        end for
    end for
    return mapping
end function
```

Question 4

Without ray tracing, we cannot compute where the projector's light lands on the surface, since we do not know the surface's geometry. Instead, we observe it: we project known patterns with the projector and record where they appear with the camera. The physical surface then performs the "intersection" for us, so the method works for a surface of any shape.

The basic idea (one pixel).

- Light up a single projector pixel $(i, j)_p$.
- Use the camera to see where the light appears. The camera pixel where it appears is $(i, j)_c$.
- This gives the mapping $(i, j)_p \mapsto (i, j)_c$.

In general, the light from one projector pixel may cover several pixels in the camera's image plane, so $(i, j)_p$ maps to a *set* of camera pixels rather than a single one.

Making it efficient. Doing this sequentially for every projector pixel would take a long time. We could do it in parallel by giving each projector pixel a distinct colour and seeing where each colour lands in the camera's image, but there are too many pixels for this to be feasible.

Instead, taking inspiration from structured light [1], we encode rows and columns separately, using two separate sets of images (one for rows, one for columns).

Choosing the colours. Let n be the maximum number of colours the camera can reliably tell apart.

Removing the background. Before projecting any patterns, we take a picture of the surface with the projector off. Then, for each pattern, we "subtract" this original image from the picture taken while the projector is projecting, so that only the light coming from the projector remains.

Procedure.

1. **Rows.** Let $m = \lceil 500/n \rceil$. Repeat the following m times, each time covering rows that were not covered previously: project a picture containing n different colours, one per row, and record which camera pixels show each colour. Together, these give the mapping

   $$\textit{rowMapping} : \text{row in projector frame} \to \text{set of pixels in camera frame.}$$

   (The camera pixels lit by one projector row need not form a row in the camera frame, since the surface can bend them.)

2. **Columns.** Similarly, using the same process for the columns, we get

   $$\textit{colMapping} : \text{column in projector frame} \to \text{set of pixels in camera frame.}$$

3. **Combining.** Since i corresponds to x (the column) and j to y (the row), the camera pixels corresponding to projector pixel $(i, j)_p$ are

   $$\textit{colMapping}[i] \cap \textit{rowMapping}[j].$$

   If this intersection is empty, the light from that projector pixel does not land anywhere the camera can see, so there is no mapping for $(i, j)_p$.

References

[1] Structured light video, `https://youtu.be/mSsnf5tqXnA`.